

Lawrenn Supabase Migration Workflow

A practical reference for moving database changes, Edge Functions, and secrets from staging to production safely.

Core principle

Do not treat staging as something you copy into production. Treat migration files in Git as the source of truth. The same approved migration should run on staging first, then production.

1. Mental Model

Lawrenn should have three separate lanes:

- Database schema: handled by SQL migration files and supabase db push.
- Supabase Edge Functions: handled by supabase functions deploy.
- Secrets and environment variables: handled by supabase secrets set or the Supabase dashboard.

Database schema = migrations + db push
Edge Functions = functions deploy
Secrets = secrets set
Frontend = Vercel/Git deployment

2. What Goes in a Migration File?

Create a migration file for database-level changes. This includes:

- Tables and columns
- Indexes and constraints
- RLS policies/rules
- Postgres SQL functions
- Triggers
- Views
- Enums/types
- Extensions

Do not use database migrations for:

- Supabase Edge Functions
- Function secrets/API keys
- Frontend app code
- Vercel environment variables
- Actual production row data unless you intentionally write seed data

3. The Standard Database Workflow

1. Create a migration file before making the database change.
2. Write the SQL change inside that migration file.
3. Link your local project to lawrenn-stg.
4. Push the migration to staging.
5. Test stg.lawrenn.com and verify the Supabase staging dashboard.
6. Commit the migration file to Git.
7. Link your local project to production.
8. Run a production dry run.
9. Push the migration to production only after the dry run looks safe.

4. Create a New Migration

From the Lawrenn project folder:

```
cd C:\Users\Kabir\Lawrenn
npx supabase migration new add_intake_tables
```

This creates a new SQL file in:

```
supabase\migrations\
```

Example filename:

```
20260615010101_add_intake_tables.sql
```

5. Write the SQL Change

Open the newest migration file and write the SQL. Example:

```
create table public.intake_requests (
  id uuid primary key default gen_random_uuid(),
  firm_id uuid not null,
  client_name text not null,
  client_email text,
  client_phone text,
  case_type text,
  description text,
  status text not null default 'new',
  created_at timestamptz not null default now()
);

alter table public.intake_requests enable row level security;
```

RLS warning

For early testing, simple policies may work. For Lawrenn production, RLS should be firm/user-scoped so one law firm can never read another firm's data.

6. Push the Migration to Staging

Link the project to your staging Supabase project:

```
npx supabase link --project-ref STAGING_REF
```

Then push the migration:

```
npx supabase db push
```

After this, check:

- stg.lawrenn.com works with the new database change.
- The new table/function/policy exists in the lawrenn-stg Supabase dashboard.
- RLS allows valid actions and blocks invalid actions.
- No unexpected errors appear in the app or Edge Function logs.

7. Commit the Migration

Once staging works, commit the migration to Git:

```
git status
git add supabase/migrations
git commit -m "Add intake requests table"
```

Why commit before production?

The migration file is your database history. If production changes are not tracked in Git, you lose the ability to understand, review, and reproduce your schema over time.

8. Push the Same Migration to Production

Link to the production Supabase project:

```
npx supabase link --project-ref PROD_REF
```

Run a dry run first:

```
npx supabase db push --dry-run
```

If the dry run looks correct, push to production:

```
npx supabase db push
```

9. Edge Functions Workflow

Edge Functions are not database migrations. They live under:

```
supabase/functions/FUNCTION_NAME\
```

Deploy to staging:

```
npx supabase functions deploy FUNCTION_NAME --project-ref STAGING_REF
```

Test staging, then deploy the same function to production:

```
npx supabase functions deploy FUNCTION_NAME --project-ref PROD_REF
```

10. Secrets Workflow

Secrets do not move through migrations or function deployments. Set them separately for staging and production.

```
npx supabase secrets set OPENAI_API_KEY=your_staging_key --project-ref STAGING_REF  
npx supabase secrets set OPENAI_API_KEY=your_prod_key --project-ref PROD_REF
```

Check secrets:

```
npx supabase secrets list --project-ref STAGING_REF  
npx supabase secrets list --project-ref PROD_REF
```

Security rule

Never paste production secrets into migration files, GitHub YAML files, frontend code, or committed .env files.

11. Full Example: Database + Edge Function

Example scenario: Lawrenn adds a case summary feature.

Database migration

```
npx supabase migration new add_case_summaries
```

```
create table public.case_summaries (
  id uuid primary key default gen_random_uuid(),
  case_id uuid not null,
  summary text not null,
  created_at timestamptz not null default now()
);

alter table public.case_summaries enable row level security;
```

Push to staging

```
npx supabase link --project-ref STAGING_REF
npx supabase db push
```

Deploy Edge Function to staging

```
npx supabase functions deploy generate-case-summary --project-ref STAGING_REF
```

Promote to production

```
npx supabase link --project-ref PROD_REF
npx supabase db push --dry-run
npx supabase db push
npx supabase functions deploy generate-case-summary --project-ref PROD_REF
```

12. Safety Checklist Before Production

- The migration was tested on lawrenn-stg.
- The migration file is committed to Git.
- The production dry run only shows the expected migration.
- There are no unexpected DROP, DELETE, TRUNCATE, or INSERT statements.
- RLS policies are production-safe and firm/user-scoped.
- Edge Function secrets are set separately for production.
- You tested the exact production user flow after deploy.

Useful checks:

```
Select-String -Path "supabase\migrations\*.sql" -Pattern "DROP TABLE"
Select-String -Path "supabase\migrations\*.sql" -Pattern "TRUNCATE"
Select-String -Path "supabase\migrations\*.sql" -Pattern "DELETE FROM"
Select-String -Path "supabase\migrations\*.sql" -Pattern "INSERT INTO"
```

13. One-Time Recovery/Snapshot Method

If you already made manual changes in staging and need to capture the schema, you can use a schema dump as a one-time recovery method. This is not the preferred long-term workflow.

```
npx supabase db dump --linked --schema public -f staging_schema.sql
```

Then inspect the dump:

```
Select-String -Path staging_schema.sql -Pattern "CREATE TABLE"  
Select-String -Path staging_schema.sql -Pattern "CREATE FUNCTION"  
Select-String -Path staging_schema.sql -Pattern "CREATE POLICY"  
Select-String -Path staging_schema.sql -Pattern "CREATE TRIGGER"
```

Use dumps carefully

A full schema dump can include objects production already has. If prod is not empty, copy only the missing/safe statements into a migration instead of blindly applying the entire dump.

14. Troubleshooting Notes from Setup

Docker Client vs Server

If docker version shows only Client and no Server, Docker Desktop is not fully running or the daemon is unreachable.

```
docker version
```

You want to see both Client and Server sections.

Docker read-only file system error

If Docker fails with read-only file system while pulling the Supabase Postgres image, clean Docker data:

```
Docker Desktop -> Troubleshoot -> Clean / Purge data -> Apply & Restart
```

Then manually pull the image:

```
docker pull public.ecr.aws/supabase/postgres:17.6.1.127
```

IPv6 direct database connection issue

If the CLI/container cannot reach db.PROJECT.supabase.co over IPv6, use the Supabase Session Pooler connection string or use db dump --linked if it works in your environment.

```
npx supabase db dump --linked --schema public -f staging_schema.sql
```

15. Future Enterprise Upgrade

When Lawrenn is ready, move this into GitHub Actions:

- Push/merge to staging branch -> apply migrations to lawrenn-stg.
- Test stg.lawrenn.com.
- Merge to main -> dry run and apply migrations to production.
- Protect main so production database changes require a pull request.

Enterprise pattern

The professional pattern is not manually changing production. It is version-controlled migrations, staging validation, production dry runs, and protected releases.

16. Quick Command Reference

Task	Command
Create migration	<code>npx supabase migration new describe_change</code>
Link staging	<code>npx supabase link --project-ref STAGING_REF</code>
Push DB to staging	<code>npx supabase db push</code>
Link production	<code>npx supabase link --project-ref PROD_REF</code>
Dry run production	<code>npx supabase db push --dry-run</code>
Push DB to production	<code>npx supabase db push</code>
Deploy Edge Function to staging	<code>npx supabase functions deploy FUNCTION_NAME --project-ref STAGING_REF</code>
Deploy Edge Function to production	<code>npx supabase functions deploy FUNCTION_NAME --project-ref PROD_REF</code>
Set secret	<code>npx supabase secrets set KEY=value --project-ref PROJECT_REF</code>
List secrets	<code>npx supabase secrets list --project-ref PROJECT_REF</code>

Final Rule

For Lawrenn, database changes should be intentional, repeatable, and version-controlled.

Create a migration, test it in staging, then apply the exact same migration to production after a dry run. Use separate commands for Edge Functions and secrets.